

The Neutrino Manual

A small functional array language

Mico Mrkaic

July 2026

Contents

1. Getting started	1
2. The REPL	2
3. Values and types	2
4. Operators and expressions	3
5. Variables and scope	4
6. Control flow	4
7. Functions	5
8. Arrays and matrices	5
9. Linear algebra	6
10. Complex numbers	7
11. Special functions and statistics	7
12. Random numbers	9
13. Plotting	9
14. Data files	10
15. Output and formatting	10
16. Scripts and tools	10
17. Builtin reference	11
18. Grammar summary	16

This manual covers Neutrino as implemented: every example below was executed against the interpreter and shows its actual output. For a quick pitch and build instructions see the [README](#); for the honest list of sharp edges see [KNOWN_LIMITATIONS](#).

1. Getting started

Build (a C23 compiler and libm; readline optional — see the README for macOS notes):

```
make          # ./neutrino, the REPL
make test     # golden suite + codegen goldens
```

The banner shows the version, when the binary was built, and when the session started. `neutrino --version` prints the same from the command line. Start the REPL and type an expression; its value echoes back:

```
neutrino> 2 + 3 * 4
14
neutrino> [1, 2; 3, 4] * [5; 6]
[ 17
  39 ]
```

A trailing `;` evaluates a statement but suppresses the echo. `#` and `%` both start a comment that runs to end of line. `Ctrl-D` exits; `Ctrl-C` cancels the current input.

2. The REPL

The interactive shell adds conveniences on top of the language. None of them apply to scripts — they are REPL features.

Commands (typed bare, not as function calls):

Command	Effect
<code>help / help(f)</code>	catalogue of all builtins / details plus usage examples for one
<code>who</code>	your variables, with type and shape
<code>clear() / clear("a", ...)</code>	remove all user variables / the named ones (builtins are safe)
<code>mem</code>	workspace size and peak process memory
<code>format ...</code>	number display: <code>format long</code> , <code>format short e</code> , <code>format(8)</code> , <code>format</code> to show
<code>pretty on\ off</code>	aligned multi-line matrix display (default on in the REPL)
<code>more on\ off</code>	page long output through <code>\$PAGER</code> (default off)
<code>!cmd</code>	run a shell command (<code>!ls</code> , <code>!git status</code>)
<code>dis(f)</code>	disassemble a function's bytecode

Every builtin's help includes executable examples with their actual output — the examples are machine-verified against the interpreter, like this manual:

```
neutrino> help(median)
median(A) | median(A, dim)
    median of all elements, or along dim
    builtin, 1 to 2 arguments
    e.g. median([1, 2, 3, 4])           % 2.5
```

Autocall. A bare name that is a zero-argument builtin or closure is called: `who`, `help`, `rand` work without parentheses, and so does your own `let f = fn -> 42; f`.

Line editing. With `readline` (or macOS `libedit`), you get history (persisted to `~/.neutrino_history`), completion on builtin and variable names, and the usual Emacs keys. Multi-line constructs continue with a `...>` prompt until the end arrives.

Display defaults. The REPL prints matrices as aligned blocks (`pretty on`) and numbers in a terse 6-significant-digit format; both are configurable — see [Output and formatting](#).

3. Values and types

Neutrino has nine value kinds. The scalar kinds:

Type	Literals	Notes
Int	42, -7	64-bit signed; overflow wraps silently (documented footgun)
Float	3.14, 1e-9, 2.5e3	IEEE double

Type	Literals	Notes
Bool	true, false	distinct from numbers: 1 == true is an error
Complex	2i, 1 + 3i, 2.5i	double re/im pair
String	"hello"	byte strings: + concatenates, comparisons are lexicographic, s[i]/s[a:b] index bytes (see the Strings section)
Null	null	the “no value” value; a suppressed or valueless statement yields it

And the compound kinds: Array (the 2-D numeric matrix — every array is rows x cols; a scalar is *not* a 1x1 array), Record ({x = 1, y = 2}, fields via .x), and Function (builtins and closures).

The type discipline is strict rather than coercive: Bool does not silently become a number in arithmetic or comparison (1 == true errors), and mixing Bool with numerics in an array literal is an error. The one deliberate blur: Int and Float compare and combine freely (5 == 5.0 is true), and / always produces a Float (4 / 2 is 2.0).

Floating point behaves like floating point:

```
neutrino> 0.1 + 0.2 == 0.3
false
neutrino> 5 / 0
inf
```

Division by zero yields inf/nan rather than an error — test with isfinite/isnan where it matters.

4. Operators and expressions

From loosest to tightest binding:

Level	Operators	Notes
pipe	\ >	left-assoc; see Functions
logical or	\	short-circuit
logical and	&&	short-circuit
elementwise or	\	on logicals/arrays
elementwise and	&	on logicals/arrays
comparison	== ~= < <= > >=	elementwise on arrays -> logical array
range	a:b, a:step:b	inclusive; float steps fine
additive	+ -	
multiplicative	* / .* ./ \ \	* is the matrix product on matrices; .* elementwise; \ left division (solve)
unary	- ! ~	binds looser than ^: -2^2 is -4
power	^ .^	right-assoc: 2^3^2 is 512; ^ on a square matrix is the matrix power
postfix	' .' f(x) a[i] .field	' is conjugate transpose, .' plain

The `.-`-prefixed operators are the elementwise family, exactly as in Octave:

```
neutrino> [1, 2, 3] .* [4, 5, 6]
[4, 10, 18]
neutrino> 2 .^ [1, 2, 3]
[2, 4, 8]
neutrino> [1i, 2]'          # conjugate transpose
[ -1i
  2+0i ]
```

Scalars broadcast against arrays in every elementwise operation (`[1, 2] + 1` is `[2, 3]`); two arrays must agree in shape exactly.

5. Variables and scope

`let name = value` is the binding **statement**: at the top level it creates a global; inside a loop body or block it creates a local scoped to that construct. A bare `name = value` (no `let`) is **assignment**: it walks outward through the enclosing scopes and updates the nearest existing name — this is how a loop body updates an accumulator defined outside it.

`let name = value in body` is the binding **expression**: name is visible only in body, and the whole thing evaluates to body. Bindings nest and shadow:

```
neutrino> let a = 1 in a + (let a = 100 in a) + a
102
```

6. Control flow

`if` is an expression:

```
neutrino> let x = 5; if x > 3 then "big" else "small" end
"big"
```

The `else` branch is optional; if the condition is false and there is no `else`, the result is `null`. Conditions must be `Bool` — a number is not a condition.

Loops are statements (they yield `null`):

```
neutrino> let s = 0; for i = 1:10 do s = s + i end; s
55
neutrino> let n = 1; while n < 100 do n = n * 2 end; n
128
```

`break` leaves the nearest loop, `continue` skips to its next iteration, and `return [value]` exits the enclosing function. All three are safe anywhere — including mid-expression (`acc + (if bad then continue else v end)`): the VM restores the loop's stack state on a non-local exit.

Block expressions sequence statements inside parentheses; the value is the final expression, and `let` bindings are local to the block:

```
neutrino> (let x = 3; let y = 4; sqrt(x*x + y*y))
5
neutrino> (let q = 7; q); q
error: undefined name 'q'          # block locals do not leak
```

This is the natural shape for a multi-step function body.

7. Functions

`fn params -> expr` is a lambda; its body is one expression (use a block expression or `let ... in` for multiple steps). Functions are values: bind them, pass them, return them.

```
neutrino> let f = fn x -> x ^ 2 + 1; f(3)
10
neutrino> let add = fn a -> fn b -> a + b; add(2)(5)    # currying
7
neutrino> let g = fn x -> (let s = x * x; s + 1); g(4)
17
```

Closures capture by value at creation. Recursion works through the function's own name.

Sections. A parenthesised expression containing `_` becomes a lambda; each `_` is a fresh parameter, left to right: `(_ + 1)` is `fn x -> x + 1`, `(_ * _)` takes two arguments.

map. `map(f, A)` applies `f` elementwise: `map((_ * 10), [1, 2, 3])` is `[10, 20, 30]`.

The pipe. `x |> rhs` feeds a value forward. If `rhs` mentions `@`, the pipe binds `@` to `x` and evaluates `rhs`; if `rhs` is a bare callable, the pipe applies it:

```
neutrino> [1, 2, 3, 4] |> sum(@) |> sqrt
3.16228
neutrino> 5 |> @ + 1
6
neutrino> 9 |> sqrt                # bare callable: sqrt(9)
3
```

Pipes chain left to right, which reads as a data-flow pipeline.

8. Arrays and matrices

Matrix literals use `,` between columns and `;` between rows; every array is two-dimensional and **1-indexed**. `[]` is the empty array.

Indexing supports scalars, ranges, `:` (everything along a dimension), `end` (the last index, with arithmetic), index vectors (gather), and logical masks:

```
neutrino> let A = [1, 2, 3; 4, 5, 6]
[ 1  2  3
  4  5  6 ]
neutrino> size(A)
[2, 3]
neutrino> A[2, 3]          # element
6
neutrino> A[1, :]          # row
[1, 2, 3]
neutrino> A[:, 2]          # column
[ 2
  5 ]
neutrino> A[end, end]
6
neutrino> let v = [10, 20, 30, 40]; v[2:3]
[20, 30]
neutrino> v[v > 15]        # logical mask
[20, 30, 40]
neutrino> v[[1, 4]]        # gather
[10, 40]
```

Indexed assignment works with the same selectors, copy-on-write: `A[1, 2] = 9`, `v[v < 0] = 0`, `A[2, :] = [7, 8, 9]`. The target must be a plain name and indices must be in range (no auto-growing).

`unique(A)` returns the sorted distinct elements — vectors keep their orientation, matrices flatten to a row. **Logical arrays** come from comparisons and drive masking and counting: `sum(A > 2)` counts, `any/all` test, `find(mask)` gives 1-based positions, `where(mask, a, b)` selects elementwise.

Construction and reshaping: `zeros`, `ones`, `eye`, `diag`, `linspace`, `reshape` (row-major), `repmat`, `ranges 1:n`. **Reductions** take an optional dimension: `sum(A, 1)` down columns, `sum(A, 2)` across rows, `max(A, [], dim)` for the extrema.

Descriptive statistics follow the same conventions. `var` and `std` normalize by $N-1$ by default (`var(A, 1)` divides by N), `median` handles even counts by averaging, and `quantile` uses linear interpolation between order statistics (the NumPy default):

```
neutrino> let x = [2, 7, 4, 9, 3]
neutrino> var(x)
8.5
neutrino> quantile(x, [0.25, 0.5, 0.75])
[3, 4, 7]
```

`cov(X)` and `corr(X)` treat a matrix's **columns as variables** and rows as observations, returning the $p \times p$ covariance / Pearson correlation matrix (`cov` takes the same w normalization as `var`). On two vectors they return the scalar: `cov(x, y)`, `corr(x, y)`. A constant column has no correlation to speak of — those entries are `nan`:

```
neutrino> corr([1, 2, 3, 4, 5], [2, 1, 4, 3, 5])
0.8
```

9. Linear algebra

`*` is the matrix product; `\` solves. Square systems use LU with partial pivoting; non-square `\` is least squares — overdetermined gives the LS fit, underdetermined the minimum-norm solution.

```
neutrino> [2, 1; 1, 3] \ [3; 5]
[ 0.8
  1.4 ]
neutrino> [1, 1; 1, 2; 1, 3] \ [1; 2; 2]      # regression: intercept, slope
[ 0.666667
  0.5 ]
```

The decompositions return records, so the pieces have names:

Call	Returns	Identity
<code>lu(A)</code>	<code>{L, U, p}</code>	$P \cdot A = L \cdot U$
<code>qr(A)</code>	<code>{Q, R}</code>	$A = Q \cdot R$
<code>chol(A)</code>	<code>L</code>	$L \cdot L' = A$ (Hermitian PD)
<code>svd(A)</code>	<code>{U, S, V}</code>	$A = U \cdot \text{diag}(S) \cdot V'$
<code>eig(A)</code>	<code>{values, vectors}</code>	$A \cdot V = V \cdot \text{diag}(\text{values})$

`eig` handles both Hermitian matrices (Jacobi; real ascending eigenvalues, orthonormal vectors) and general ones (complex QR + inverse iteration; complex pairs come out right). All of it is complex-capable — `'` being the conjugate transpose is what makes the identities hold. `kron(A, B)` is the Kronecker product, `det`, `inv`, `trace`, `norm`, `dot` round out the set.

10. Complex numbers

The imaginary literal is a number followed by `i`. Complex values propagate through arithmetic and the math library; results stay complex (`1i * 1i` is `-1+0i`, not `-1`). `real`, `imag`, `conj`, `angle` access the parts, `abs` is the modulus. Functions with limited real domains return complex off it: `sqrt(-4)` is `2i`, `log(-1)` is `3.14159i`, `asin(2)` is complex.

11. Special functions and statistics

The special-function set is chosen as *primitives* from which the classical distributions are one-liners:

```
neutrino> let normcdf = fn x -> 0.5 * erfc(-x / sqrt(2))
neutrino> normcdf(1.96)
0.975002
neutrino> norminv(0.975)
1.95996
```

`gammainc(x, a)` is the regularized lower incomplete gamma — the chi-square CDF is `gammainc(x/2, k/2)`. `betainc(x, a, b)` is the regularized incomplete beta — Student-t and F CDFs fall out of it. `erf/erfc`, `beta/lbeta`, `gamma/lgamma`, `digamma`, and integer-order Bessel `besselj/bessely` complete the set. All are real-domain, elementwise over arrays, and validated against SciPy.

Root finding, minimization, integration

These three take a **function argument** — a closure or builtin — and call it repeatedly. `fzero(f, a, b)` finds a root by Brent's method (`f(a)` and `f(b)` must differ in sign); `fminbnd(f, a, b)` minimizes on an interval and returns `{x, fx}`; `integral(f, a, b[, tol])` integrates adaptively (Simpson with Richardson estimate; finite limits; default tolerance `1e-10`). Closures capture data, so parameterized problems read naturally — a bond's yield to maturity:

```
neutrino> let cf = [100, 100, 100, 1100]
neutrino> let npv = fn r -> sum(cf ./ ((1 + r) .^ (1:4))) - 1000
neutrino> format(6); fzero(npv, 0.01, 0.3)
0.100000
```

A divergent or wildly oscillatory integrand fails with an error rather than a wrong answer; infinite limits are rejected — substitute to a finite domain first.

Strings

Strings are byte sequences (UTF-8 passes through but is not interpreted; length and indexing count bytes). `+` concatenates; `==`, `!=`, `<` and friends compare lexicographically, shorter prefixes first; indexing uses the same machinery as arrays, so ranges, end, and even permutations work:

```
neutrino> let s = "neutrino"
neutrino> s[1:3] + "!" + s[end]
"neu!o"
neutrino> "apple" < "banana"
true
```

The library: `upper`, `lower`, `trim`, `contains(s, sub)`, `startswith(s, p)`, `endswith(s, p)`, `str-rep(s, old, new)`. Two bridges: `str(x)` gives any value's display text as a string, `num(s)` parses a string as a number (Int if exact, else Float). And `fmt(tmpl, ...)` is print's template engine returning a string instead of printing:

```
neutrino> fmt("run {} done in {:.2f}s", 3, 1.234)
"run 3 done in 1.23s"
```

Strings also form **arrays**: `["a", "bb"; "c", "dd"]` is a 2x2 String matrix (mixing strings with numbers in one matrix is an error). Indexing, slicing, `end`, transpose, reshape, sort, and unique all work, and elementwise operations do what a data-frame user hopes:

```
neutrino> let names = ["si", "at", "de", "si"]
neutrino> names == "si"
[true, false, false, true]
neutrino> names[names == "si"]
["si", "si"]
neutrino> ["pre_", "un_"] + "fix"
["pre_fix", "un_fix"]
```

Numeric reductions (`min`, `max`, `sum`, `norm`, ...) refuse string arrays rather than computing nonsense, and assignment cannot mix kinds: a String cell never silently becomes a number, nor the reverse.

`strsplit(s, sep)` and `strjoin(a, sep)` convert between strings and string arrays; `fields(r)` returns a record's field names as a string column; `readtable` loads non-numeric CSV columns as string arrays (quoted cells, RFC-4180 style, are handled); `writescv` writes string matrices with proper quoting; and plots take `{labels = ["low", "high"]}` for multi-series legends alongside the older `label1, label2, ...` form.

String-and-number arithmetic is still an error — there is no implicit conversion in either direction; use `str` and `num` to cross the bridge.

Packages: load

`load("file.nu")` runs a file in the current session; its `let` bindings persist afterwards. A package is just a file of definitions — and a record of closures makes a namespace, so packages don't collide:

```
neutrino> load("tests/data/mathlib.nu")
neutrino> cube(4)
64
neutrino> geo.hyp([3, 4])
5
```

`save("ws.nu")` writes the whole workspace — every variable and function — as reloadable source; restore it with `load("ws.nu")`. The file is plain Neutrino, so it is readable and editable. Functions that capture variables (closures made by other functions) cannot be serialized and refuse with a clear message; a failed save leaves no file behind. `body(f)` prints the source of a user-defined function:

```
neutrino> let cube = fn x -> x^3
neutrino> body(cube)
fn x -> x^3
```

Expressions may span lines wherever a `(`, `[`, or `{` is open — newlines there are plain whitespace (matrix rows still take an explicit `;`). At the top level a newline ends the statement, and the REPL reads continuation lines automatically while a bracket is open.

Packages validate their inputs with `error` and `assert`:

```
neutrino> let f = fn x -> (assert(x > 0, "f needs x > 0, got {}", x); sqrt(x))
neutrino> f(-4)
error: f needs x > 0, got -4
```

Packages can load other packages (nesting is capped, so circular loads error cleanly). Closures capture by value, so packages are libraries of functions rather than stateful modules. Errors inside a loaded file are reported with the file name; a parse error includes its line and column within the file.

12. Random numbers

The generator is xoshiro256** seeded through splitmix64, and it is **reproducible by default**: every fresh session starts from the same fixed seed, so a script's random draws are stable run to run. `rng(seed)` reseeds — same seed, same stream. `rand`, `randn`, `randi` draw uniform, normal, and integer variates, scalar or matrix-shaped (`rand(3)`, `randn(2, 4)`).

13. Plotting

Plotting is delegated to **gnuplot**, out of process — a soft dependency: the language works without it, and `plot` reports cleanly if it is missing (`plot: gnuplot failed (exit 127) – is gnuplot installed?`).

`plot(y)` plots a vector against its index; `plot(x, y)` plots pairs; if `y` is a **matrix**, each column is a separate series. An optional trailing argument is either a gnuplot style string ("points", "lines lw 2", "impulses") or an options record:

```
neutrino> let x = linspace(0, 10, 200)
neutrino> plot(x, map(sin, x), {title = "sin(x)", xlabel = "x", grid = true})
```

Recognised options: `title`, `xlabel`, `ylabel`, `style` (strings); `logx`, `logy`, `grid` (booleans); `xrange`, `yrange` ([`lo`, `hi`] vectors) to fix an axis instead of letting gnuplot choose; and legend labels — `label` for a single series, `label1`, `label2`, ... for several (unlabeled series keep the series `k` default):

```
neutrino> let t = (linspace(0, 6.28, 100))';
neutrino> plot(t, [map(sin, t), map(cos, t)], {label1 = "sin", label2 = "cos"})
``` `hist(y)` draws a histogram
(`hist(y, nbins)` to choose the bin count; the default follows Sturges' rule),
and accepts the same trailing options record:
neutrino> rng(7) neutrino> hist(randn(1, 5000), 40)
```

A word of caution that ``yrange`` exists to address: gnuplot auto-ranges the y-axis to the data, which can make pure sampling noise look like structure. A histogram of 100 000 uniform draws in 20 bins has bin counts of 5000 +/- 69 (one binomial standard deviation) — auto-ranged, that +/-1.4% wiggle fills the whole plot and looks alarming; anchored at zero it is the flat wall it should be:

```
neutrino> hist(rand(1, 100000), 20, {yrange = [0, 6000]})
```

Plots open in a gnuplot window that outlives the command (``gnuplot -persist``). For scripted rendering, two environment variables redirect output — ``NEUTRINO_PLOT_TERM`` sets the gnuplot terminal and ``NEUTRINO_PLOT_OUT`` the file:

```
```sh
NEUTRINO_PLOT_TERM="pngcairo size 800,500" NEUTRINO_PLOT_OUT=fig.png \
  ./neutrino script.nu
```

(NEUTRINO_PLOT_TERM="dumb size 76,20" draws ASCII plots straight into the terminal, which is occasionally exactly what you want.) Complex data is rejected — plot `real(z)`, `imag(z)`, or `abs(z)` explicitly.

14. Data files

`readcsv(file)` reads a numeric CSV into a Float matrix; `writcsv(file, A)` writes one at full precision, so values **round-trip bit-exactly**. Empty cells become `nan` (missing data), Windows line endings are tolerated, and `{delim = ";", skip = n}` options handle other separators and preamble lines. Ragged rows and non-numeric cells are errors that name the row and column.

`readtable(file)` reads a CSV whose first line is a header and returns a **record of column vectors**, keys sanitized from the column names ("GDP Growth (%) becomes `gdp_growth`; duplicates get `_2`, `_3`, ...). This is Neutrino's data frame — named columns plus the existing mask machinery:

```
neutrino> writcsv("/tmp/m.csv", [1, 2; 3, 4]); readcsv("/tmp/m.csv")
[ 1  2
  3  4 ]

neutrino> let d = readtable("tests/data/macro.csv")
neutrino> mean(d.gdp_growth)
1.93333
neutrino> d.cpi[d.year >= 2021]
[ nan
   8 ]
```

For a headerless file use `readcsv` — `readtable` will take the first line as a header regardless of its content. A column containing text (country names, tickers) is rejected with an error naming the column: representing it needs first-class strings, which the language does not yet have.

15. Output and formatting

`format` controls how numbers print. Explicitly chosen formats keep trailing zeros for consistent width; the startup default is terse:

```
neutrino> format(4)
neutrino> for i = 1:3 do print(sqrt(i)) end
1.000
1.414
1.732
```

`print` takes either plain values (space-separated) or a template whose `{}` holes are filled in order; a hole can carry its own spec `{:[-][width][.prec][f|e|g]}` for per-hole width, precision, and conversion:

```
neutrino> print("sqrt({}) = {:8.4f}", 2, sqrt(2))
sqrt(2) = 1.4142
```

Strings print without quotes in `print`; the REPL echo shows them quoted (the echo is a representation, `print` is output). `pretty on|off` toggles the aligned matrix display; `more on` pages long output.

16. Scripts and tools

`neutrino file.nu` runs a script (top level is a statement sequence; `#/%` comments). The binary also exposes the compiler pipeline:

Invocation	Shows
<code>neutrino --tokens file.nu</code>	the token stream
<code>neutrino --ast file.nu</code>	the parse tree
<code>neutrino --dis file.nu</code>	the compiled bytecode, statement by statement

`tic` starts a monotonic wall-clock timer and `toc()` returns the elapsed seconds — bare `toc` as a statement echoes it, but in an expression position write `toc()` (a bare name is a value reference, as with any function):

```
neutrino> tic; let A = randn(100, 100); let B = A * A; toc() < 60
true
```

`vmtest` is the headless driver used by the test suite: it reads `stdin` line by line and echoes each result, so `printf 'sum(1:100)\n' | ./vmtest` prints 5050. `dis(f)` disassembles from inside the language. The golden suite (`make test`) and its ASan twin (`make test-asan`) are how changes prove themselves; `tests/dis/` pins the emitted bytecode for core constructs.

17. Builtin reference

Generated from the interpreter's own documentation table (the same data `help` shows), so it cannot drift from the implementation.

Core & introspection

Signature	Description
<code>print(...)</code> <code>print(tmpl, ...)</code>	print values; template fills { } in order; <code>{:}[w][.p][f]</code>
<code>fields(r)</code>	the record's field names, as a string column
<code>error(msg)</code> <code>error(tmpl, ...)</code>	raise a runtime error (fmt-style template)
<code>assert(cond)</code> <code>assert(cond, tmpl, ...)</code>	error unless <code>cond</code> is true
<code>who</code>	list the variables you have defined (name, type, shape)
<code>help / help(f)</code>	help lists every builtin; <code>help(f)</code> describes one
<code>system(cmd)</code>	run a shell command string; return its exit status
<code>dis(f)</code>	disassemble a function's bytecode (compiler/VM introspection)
<code>format / format(m)</code>	show or set number display: "short", "long", "short e", or a digit count
<code>size(x)</code>	[rows, cols] of <code>x</code> (a scalar is 1x1)
<code>length(x)</code>	longest dimension of <code>x</code> (0 if empty)
<code>numel(x)</code>	number of elements (rows*cols)
<code>save("file.nu")</code>	write all variables and functions as reloadable source (restore with <code>load</code>)
<code>body(f)</code>	print the source of a user-defined function
<code>load("file.nu")</code>	run a file in the current session; its let-bindings persist (a record of closures makes a module)
<code>clear()</code> <code>clear("a", ...)</code>	remove all user variables, or the named ones (builtins are untouchable)

Signature	Description
<code>mem</code>	print workspace size (variables) and peak process memory
<code>tic</code>	start the wall-clock timer (monotonic)
<code>toc</code>	seconds elapsed since tic

Strings

Signature	Description
<code>upper(s)</code>	uppercase (ASCII bytes)
<code>lower(s)</code>	lowercase (ASCII bytes)
<code>trim(s)</code>	strip leading and trailing whitespace
<code>contains(s, sub)</code>	true if sub occurs in s
<code>startswith(s, p)</code>	true if s begins with p
<code>endswith(s, p)</code>	true if s ends with p
<code>strrep(s, old, new)</code>	replace every occurrence of old with new
<code>str(x)</code>	the display text of any value, as a string
<code>num(s)</code>	parse a string as a number (Int if exact, else Float)
<code>fmt(tmpl, ...)</code>	print's template, returned as a string instead of printed
<code>strsplit(s, sep)</code>	split a string on a separator, giving a string row vector
<code>strjoin(a, sep)</code>	join a string array with a separator

Solvers

Signature	Description
<code>fzero(f, a, b)</code>	root of f in [a, b] (Brent; f(a), f(b) must differ in sign)
<code>fminbnd(f, a, b)</code>	minimum of f on [a, b] (Brent) -> {x, fx}
<code>integral(f, a, b[, tol])</code>	definite integral (adaptive Simpson, finite limits; default tol 1e-10)

Data files

Signature	Description
<code>readcsv(file[, opts])</code>	numeric CSV -> Float matrix; empty cells are nan; opts: {delim, skip}
<code>writcsv(file, A[, opts])</code>	matrix -> CSV, full precision (round-trips); opts: {delim}
<code>readtable(file[, opts])</code>	CSV with a header -> record of column vectors named from the header

Plotting

Signature	Description
<code>plot(y) plot(x, y) plot(x, Y, opts)</code>	line plot via gnuplot; Y columns are series; opts: style string or {title, xlabel, ylabel, style, logx, logy, grid, xrange, yrange, label, label1..labelN}
<code>hist(y[, nbins][, opts])</code>	histogram via gnuplot; opts as in plot (yrange to anchor the axis, label for the legend)

Array construction

Signature	Description
<code>zeros(r, c)</code>	r-by-c matrix of zeros
<code>ones(r, c)</code>	r-by-c matrix of ones
<code>eye(n)</code>	n-by-n identity matrix
<code>diag(x)</code>	vector -> diagonal matrix; matrix -> its diagonal as a column
<code>linspace(a, b, n)</code>	row of n points evenly spaced from a to b inclusive
<code>reshape(A, r, c)</code>	reinterpret A's elements as r-by-c (row-major), element count must match
<code>repmat(A, m, n)</code>	tile A into an m-by-n grid of copies

Reductions

Signature	Description
<code>sum(A) sum(A, dim)</code>	sum of all elements, or along dim (1 = columns, 2 = rows)
<code>prod(A) prod(A, dim)</code>	product of all elements, or along dim
<code>cov(X[, w]) cov(x, y[, w])</code>	covariance matrix of X's columns (rows = observations), or scalar cov of two vectors; w as in var
<code>corr(X) corr(x, y)</code>	Pearson correlation matrix of X's columns, or scalar correlation of two vectors
<code>var(A) var(A, w) var(A, w, dim)</code>	variance; w = 0 divides by N-1 (default), w = 1 by N
<code>std(A) std(A, w) std(A, w, dim)</code>	standard deviation (sqrt of var, same normalization)
<code>median(A) median(A, dim)</code>	median of all elements, or along dim
<code>quantile(x, p)</code>	quantile(s) of the data at probability p (scalar or vector); linear interpolation
<code>mean(A) mean(A, dim)</code>	mean of all elements, or along dim
<code>min(A) min(a, b) min(A, [], dim)</code>	smallest element; elementwise min; or min along dim
<code>max(A) max(a, b) max(A, [], dim)</code>	largest element; elementwise max; or max along dim
<code>any(mask) any(mask, dim)</code>	true if any element is nonzero/true (overall or along dim)
<code>all(mask) all(mask, dim)</code>	true if every element is nonzero/true (overall or along dim)

Array utilities

Signature	Description
<code>unique(A)</code>	sorted distinct elements; vectors keep orientation, matrices flatten to a row
<code>sort(A)</code>	ascending sort: a vector as a whole, a matrix by column
<code>find(mask)</code>	1-based positions of nonzero/true elements (row-major)
<code>where(mask) where(mask, a, b)</code>	indices of true, or pick a where true and b where false
<code>cumsum(A)</code>	cumulative sum along a vector, or down each column
<code>cumprod(A)</code>	cumulative product along a vector, or down each column
<code>diff(A)</code>	consecutive differences along a vector, or down each column
<code>flipud(A)</code>	reverse row order (flip up-down)
<code>fliplr(A)</code>	reverse column order (flip left-right)

Mathematical functions

Signature	Description
<code>abs(x)</code>	absolute value, or complex magnitude
<code>sqrt(x)</code>	square root (complex result for negative reals)
<code>cbrt(x)</code>	real cube root
<code>exp(x)</code>	e raised to the x (complex-aware)
<code>log(x)</code>	natural logarithm (complex for negatives)
<code>ln(x)</code>	natural logarithm (alias for log)
<code>log10(x)</code>	base-10 logarithm (complex for negatives)
<code>log2(x)</code>	base-2 logarithm (complex for negatives)
<code>sign(x)</code>	-1 / 0 / +1 by sign; z/
<code>floor(x)</code>	round toward -infinity (componentwise on complex)
<code>ceil(x)</code>	round toward +infinity (componentwise on complex)
<code>round(x)</code>	round to nearest (componentwise on complex)
<code>trunc(x)</code>	round toward zero
<code>hypot(a, b)</code>	$\sqrt{a^2 + b^2}$ without overflow (elementwise)
<code>mod(a, b)</code>	modulo, result takes the sign of b (elementwise)
<code>rem(a, b)</code>	remainder, result takes the sign of a (elementwise)
<code>gamma(x)</code>	gamma function (real, elementwise)
<code>erf(x)</code>	error function (real, elementwise)
<code>erfc(x)</code>	complementary error function $1 - \text{erf}(x)$
<code>beta(a, b)</code>	beta function ($a, b > 0$, elementwise)
<code>lbeta(a, b)</code>	log of the beta function

Signature	Description
<code>gammainc(x, a)</code>	regularized lower incomplete gamma $P(a, x)$ (the χ^2 CDF)
<code>betainc(x, a, b)</code>	regularized incomplete beta $I_x(a, b)$ (Student-t / F CDFs)
<code>norminv(p)</code>	standard normal quantile (inverse CDF)
<code>digamma(x)</code>	digamma $\psi(x) = d/dx \log \Gamma(x)$
<code>besselj(n, x)</code>	Bessel function of the first kind, integer order n
<code>bessely(n, x)</code>	Bessel function of the second kind, integer order n ($x > 0$)
<code>lgamma(x)</code>	log of

Linear algebra

Signature	Description
<code>kron(A, B)</code>	Kronecker product: $(m \times n) \text{ kron } (p \times q) \rightarrow (mp \times nq)$
<code>dot(a, b)</code>	inner product of two vectors
<code>norm(x) norm(x, p)</code>	vector p -norm ($p = 1$ or 2 , default 2); matrix Frobenius norm
<code>trace(A)</code>	sum of the diagonal
<code>det(A)</code>	determinant via LU
<code>inv(A)</code>	matrix inverse (solves $A \cdot I$)
<code>lu(A)</code>	LU with partial pivoting $\rightarrow \{L, U, p\}$, so $PA = LU$
<code>qr(A)</code>	Householder QR $\rightarrow \{Q, R\}$ (real or complex)
<code>chol(A)</code>	Cholesky factor L (lower), $L^*L' = A$ (SPD / Hermitian PD)
<code>eig(A)</code>	eigendecomposition $\rightarrow \{\text{values}, \text{vectors}\}$; Hermitian (ascending real) or general (complex)
<code>svd(A)</code>	thin SVD $\rightarrow \{U, S, V\}$, $A = U \text{diag}(S) V'$ (S descending)

Trigonometric & hyperbolic

Signature	Description
<code>sin(x)</code>	sine (complex-aware, elementwise)
<code>cos(x)</code>	cosine (complex-aware, elementwise)
<code>tan(x)</code>	tangent (complex-aware, elementwise)
<code>asin(x)</code>	arcsine (complex outside $[-1, 1]$)
<code>acos(x)</code>	arccosine (complex outside $[-1, 1]$)
<code>atan(x)</code>	arctangent (complex-aware)
<code>atan2(y, x)</code>	two-argument arctangent (elementwise)
<code>sinh(x)</code>	hyperbolic sine (complex-aware)
<code>cosh(x)</code>	hyperbolic cosine (complex-aware)
<code>tanh(x)</code>	hyperbolic tangent (complex-aware)
<code>asinh(x)</code>	inverse hyperbolic sine (complex-aware)
<code>acosh(x)</code>	inverse hyperbolic cosine (complex below 1)

Signature	Description
<code>atanh(x)</code>	inverse hyperbolic tangent (complex outside (-1, 1))

Complex accessors

Signature	Description
<code>real(z)</code>	real part (elementwise)
<code>imag(z)</code>	imaginary part (elementwise)
<code>conj(z)</code>	complex conjugate (elementwise)
<code>angle(z)</code>	argument <code>atan2(im, re)</code> (elementwise)
<code>arg(z)</code>	argument <code>atan2(im, re)</code> (alias for <code>angle</code>)

Random numbers

Signature	Description
<code>rng(seed)</code>	reseed the generator (xoshiro256**); same seed, same stream
<code>rand()</code> <code>rand(n)</code> <code>rand(r, c)</code>	uniform draws on [0, 1)
<code>randn()</code> <code>randn(n)</code> <code>randn(r, c)</code>	standard-normal draws
<code>randi(imax[, r, c])</code> <code>randi([lo, hi], ...)</code>	uniform random integers

Predicates

Signature	Description
<code>isnan(x)</code>	elementwise test for NaN -> logical
<code>isinf(x)</code>	elementwise test for +/-Inf -> logical
<code>isfinite(x)</code>	elementwise test for a finite value -> logical

Higher-order functions

Signature	Description
<code>map(f, A)</code>	apply <code>f</code> to each element of <code>A</code> , returning an array of results

18. Grammar summary

Reserved words: `let` `in` `fn` `if` `then` `else` `end` `true` `false` `null` `for` `while` `do` `break` `continue` `return`.

```

program    := statement*
statement  := 'let' NAME '=' expr                # binding (global at top level)
            | NAME '=' expr                      # assignment (walks scopes)
            | lvalue '[' indices ']' '=' expr
            | 'for' NAME '=' expr 'do' statement* 'end'
            | 'while' expr 'do' statement* 'end'
            | 'break' | 'continue' | 'return' expr?

```

```

      | expr
expr  := 'let' NAME '=' expr 'in' expr
      | 'if' expr 'then' expr ('else' expr)? 'end'
      | 'fn' NAME* '->' expr
      | '(' statement (';' statement)* ')'      # block expression
      | expr binop expr | unop expr | expr postfix
      | NAME | literal | '[' rows ']' | '{' fields '}'
      | expr '|>' expr
postfix := '"' | "."'" | '(' args ')' | '[' indices ']' | '.' NAME

```

Statements separate by newline or ; (a trailing ; also suppresses the REPL echo). Comments run from # or % to end of line.

Every example in this manual was executed against the current interpreter; the builtin reference is generated from the source. If you find a discrepancy, that is a bug — please report it.